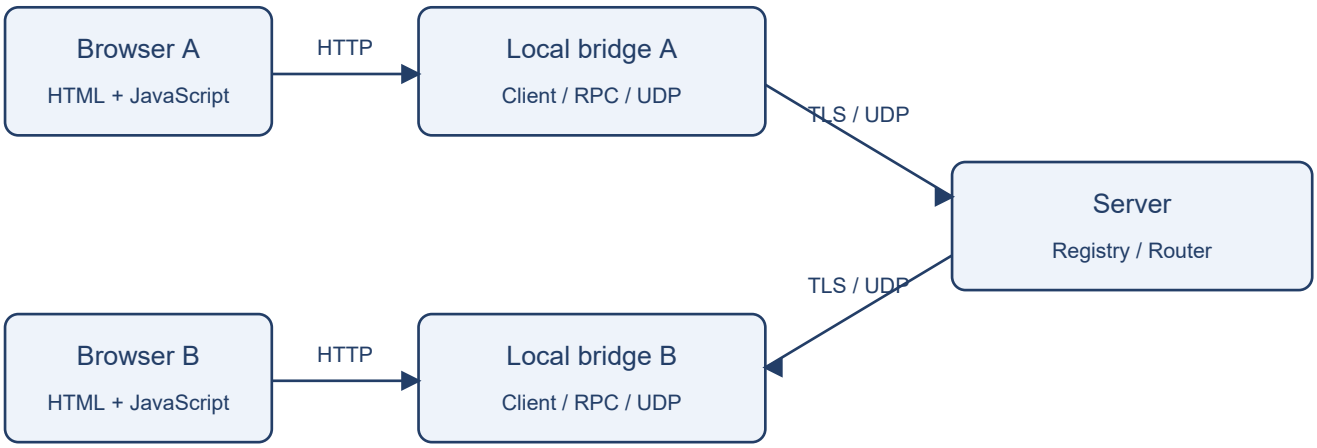
Báo cáo kỹ thuật: Distributed Streaming Collaboration

Bản tích hợp backend v2 và Web UI • 10/11/2026

Mục tiêu: ứng dụng cộng tác qua socket, hỗ trợ khám phá peers, chat riêng/chung, truyền file UDP tin cậy, mã hóa và xử lý nhiều client. Báo cáo phản ánh bản tích hợp trên nhánh feature/tcp_udp_testing, với giao diện HTML từ main và bridge mới.

1. Kiến trúc hệ thống

Trình duyệt giao tiếp HTTP localhost với Python bridge. Mỗi web session sở hữu một CollaborationClient; bridge dùng API mạng và không chạy vòng input CLI. Server trung tâm quản lý username/IP/port, chuyển tiếp yêu cầu tới người nhận. TLS 1.3 bảo vệ kênh điều khiển; AES-256-GCM bảo vệ từng datagram UDP.



Các lớp tách biệt: protocol.py đóng gói; security.py cấp chứng thư và mã hóa; reliable_udp.py giữ sequence/window/reassembly; congestion.py quản lý cwnd/pacing; application.py tương quan request/result; file_transfer.py ghi/hash file; client.py/server.py điều phối socket; web_ui là adapter và JavaScript giao diện.

Server dùng một vòng selectors cho TCP/TLS và nhận UDP; bốn worker UDP có hàng đợi 512 gói/worker xử lý phiên cố định. Tám worker ứng dụng và tám worker chuyển tiếp được chia sẻ giữa các phiên. ACK/result được xử lý độc lập với request đang chờ. Client vẫn có reader/worker riêng. TCP dở dang có timeout và không chặn peer khác.

Trạng thái kết nối: TLS_HANDSHAKE → READY_FOR_SYN → READY_FOR_AUTH → ACTIVE → CLOSED. Tên trùng bị từ chối. Cleanup chỉ xóa bản ghi do chính socket đó sở hữu.

2. Đặc tả giao thức

Header nhị phân big-endian **!BBHIIHHII**, tổng 24 bytes. Payload wire tối đa 1376 bytes, toàn datagram ứng dụng tối đa 1400 bytes, chưa tính IP/UDP header.

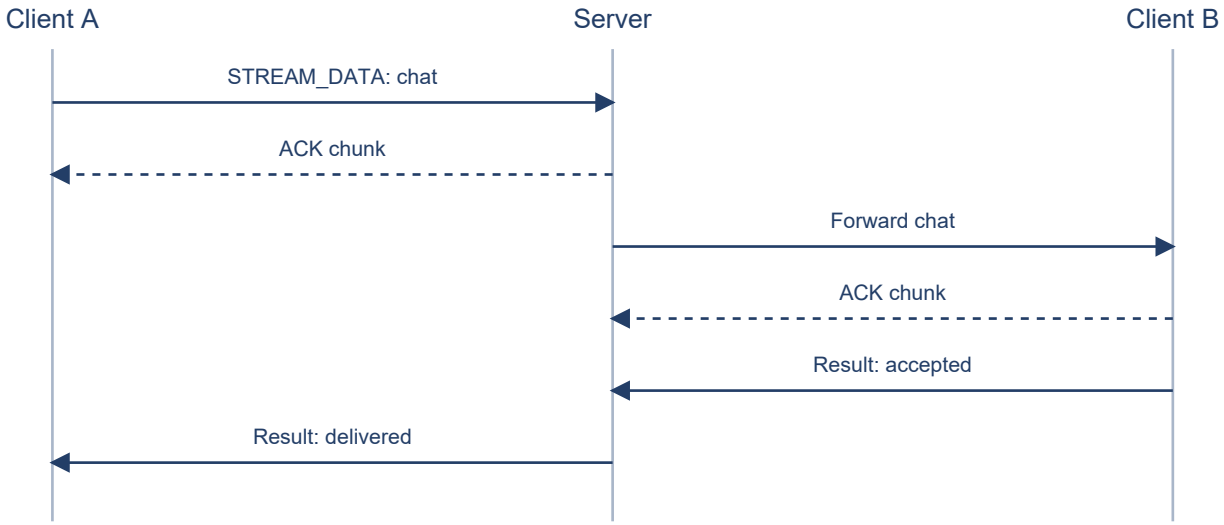
Trường	Bytes	Ý nghĩa
msg_type	1	SYN/ACK, AUTH, HEARTBEAT, PEER_UPDATE, STREAM_DATA, ACK, DISCONNECT
flags	1	EOM=1, ACK=2, ENC=4; BRD/NACK là reserved
stream_id	2	0 control; 1..65535 stream dữ liệu, tăng theo phiên/hướng
seq_num	4	Chỉ số chunk từ 0 trong stream
ack_num	4	ACK đúng một chunk; handshake dùng seq+1
win_size	2	Bề rộng cửa sổ nhận theo sequence
payload_len	2	Độ dài payload wire
timestamp	4	Milliseconds modulo 2^32, thông tin phụ
checksum	4	CRC32 của header CRC=0 và payload wire

UDP envelope: session ID 32B + counter 8B + ciphertext + GCM tag 16B. Header và ID/counter được authenticated bằng AAD. Payload DATA trước mã hóa có session ID 32B, total_chunks 4B, tối đa 1284B dữ liệu. Chỉ chunk cuối có EOM. ACK xác nhận chính xác seq; không phải cumulative ACK. CRC kiểm tra lỗi ngẫu nhiên, GCM chống sửa gói.

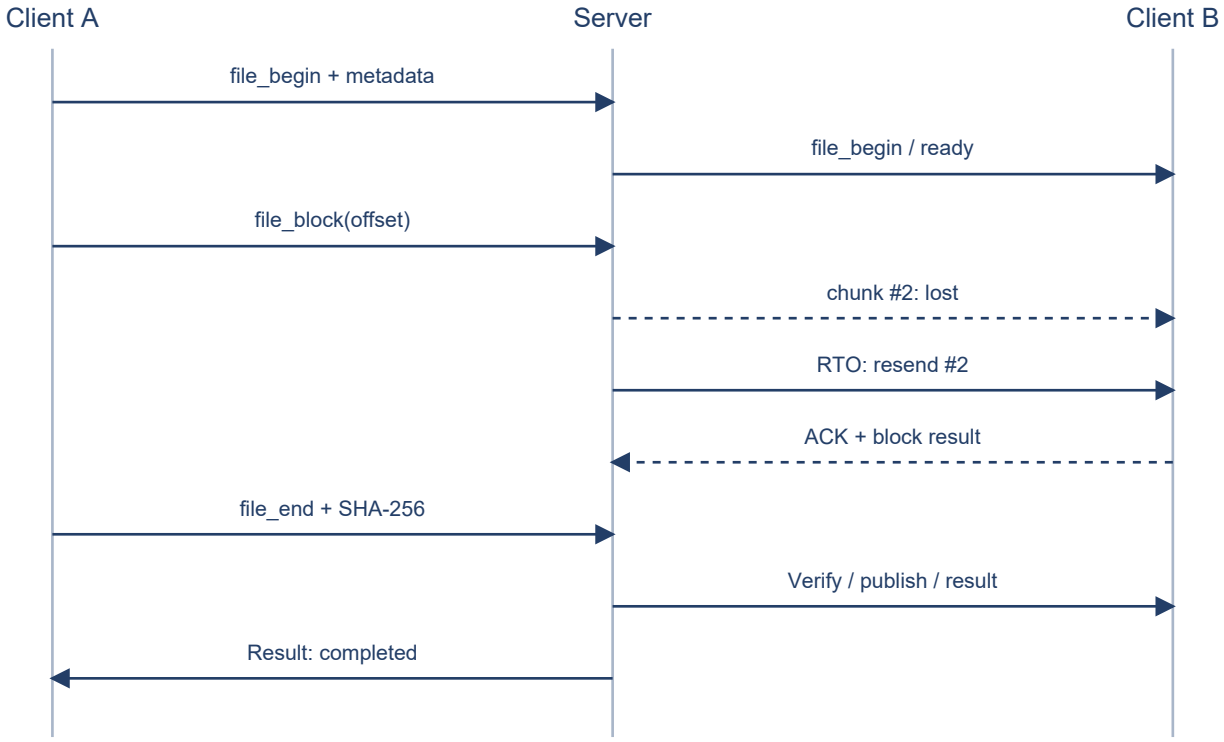
Server sinh secret 32B ngẫu nhiên và trao trong TLS đã xác minh chứng thư/hostname. HKDF-SHA256 tạo khóa AES riêng c2s/s2c và mỗi epoch 65536 datagram. Counter không reset khi retransmit; replay window 4096 counter. Phiên tối đa một giờ hoặc hết ngân sách counter thì phải reconnect. Không hardcode khóa dùng chung.

Sliding window thực: next < base + min(window local, window remote), đồng thời tổng pending của mọi stream không vượt cwnd. Selective retransmission chỉ gửi lại chunk chưa ACK. RTO thích nghi và backoff; stream có retry/timeout/receipt riêng. CUBIC-style có slow start, cubic/Reno-friendly growth, giảm cwnd khi timeout, pacing với burst tối đa tám DATA và chia lịch round-robin; không tự nhận là toàn bộ TCP CUBIC chuẩn. Worker được đánh thức khi có dữ liệu mới; ACK mở cửa sổ gửi ngay.

3. Vòng đời chat và truyền file



Unicast gửi tới một username. Broadcast chọn snapshot peers khác sender và chuyển mỗi bản qua transport tin cậy. Server lấy sender từ phiên, không tin JSON sender. Result liệt kê delivered/failed; delivered nghĩa là vào inbox phía nhận, chưa chắc người dùng đã đọc. Không dùng stream ID 100 cố định cho mọi người gửi.



Mỗi file dùng UUID riêng. Sender đọc block 256 KiB, đợi application ACK đúng offset rồi mới gửi block tiếp theo. Receiver ghi file tạm và SHA-256 tăng dần; file_end phải khớp tổng bytes/hash trước khi fsync và publish không ghi đè. Server relay từng block, không lưu toàn file. Python API giới hạn 1 GiB/file; HTTP adapter giới hạn 16 MiB/upload do sử dụng base64.

Tên lưu có UUID để tránh trùng, nhưng metadata và tên tải xuống là basename gốc. Lỗi checksum/offset, abort, disconnect hoặc idle timeout xóa file tạm. UI tiến độ lấy số bytes receiver đã ACK; trạng thái verifying tách khỏi done. Một polling loop cập nhật peers/chat/jobs, không có các bước phần trăm cứng.

4. Challenges & Solutions

Mất, trùng và đảo gói UDP: TCP không bảo vệ datagram UDP. Giải pháp là sequence riêng cho từng stream, buffer out-of-order, ACK từng chunk, RTO và retransmission chọn lọc. Stream hoàn tất giữ tombstone để ACK lại khi ACK cuối bị mất, không giao message hai lần. Nếu tất cả kết quả cuối mất, sender có thể timeout dù receiver đã nhận; ứng dụng không tự retry để tránh nhân đôi thao tác.

Nhầm ACK giữa nhiều người gửi: bản main dùng stream cố định dễ gây cross-talk. Backend v2 cấp ID tăng theo từng phiên/hướng, kiểm tra endpoint/session và mã hóa ACK; không chỉ tra một số stream chung toàn server. Test Alice/Carol cùng gửi Bob kiểm tra mỗi tin đúng một bản; broadcast cũng phải retry ở cả hai chặng.

Nghẽn và tài nguyên hữu hạn: gửi mọi chunk ngay lập tức có thể tràn buffer. Cửa sổ sequence hạn chế dữ liệu trước chỗ thiếu; cwnd chung và pacing kiểm soát đường gửi, queue và số stream bị giới hạn. Full inbox trả lỗi hoặc backpressure. Ngân sách payload là 64 MiB toàn server, 32 MiB/phiên, giữ cả khi message chờ xử lý; không phải trần RAM tiến trình. File lớn chia block, hash và ghi đĩa tăng dần. Memoryview giảm sao chép; mã hóa trực tiếp Packet bỏ bước CRC plaintext trung gian. CRC wire vẫn được kiểm tra. Control frame gộp khi ghi TLS, ACK vẫn đi UDP riêng.

Race khi đăng nhập/ngắt: registry có lock, từ chối tên trùng và cleanup kiểm tra socket sở hữu. Peer update có thứ tự trên TCP. File temp chỉ bị gỡ khỏi active sau khi đóng/xóa xong, tránh báo đã cleanup khi file vẫn tồn tại.

Mã hóa và nonce: dùng thư viện cryptography, TLS trao secret, HKDF tách hướng/epoch, nonce counter không tái sử dụng kể cả retry. Không tắt hostname verification. Khóa riêng không vào Git. Đây là hop-by-hop encryption; server thấy plaintext và username registration chưa phải xác thực danh tính bằng mật khẩu.

Bridge/UI: gọi CLI input trong worker khiến dịch vụ chết khi stdin đóng. Bridge hiện dùng API connect/authenticate và background collectors; upload job chạy riêng để HTTP status không bị chặn. Dùng textContent, đường download gắn session, kiểm tra tên file và không public thư mục filesystem. HTTP chỉ bind loopback; LAN chạy bridge tại từng máy và nối TLS tới server. Chưa triển khai HTTPS bridge.

5. Kiểm thử và vận hành

Các kiểm thử backend bao phủ 20–40 client đồng thời, TLS trust, replay/rotation, sliding window/CUBIC, chat/file, disconnect và mô phỏng loss/reorder/duplicate. Test HTTP thật kiểm tra login trùng, concurrent DM, broadcast mất DATA/ACK, upload/hash/download đúng quyền, progress bị chặn tại đúng ACK khối và tiến trình bridge với stdin=DEVNULL. Thêm kiểm thử TCP dở dang, budget bộ nhớ và đầy send buffer. README ghi đúng web_ui/server_web.py và requirements.txt.

Kiểm thử trình duyệt Edge headless mở hai phiên Alice/Bob: đăng nhập, live peers, DM/broadcast, hiển thị text chứa thẻ HTML an toàn, truyền file 1 MiB, progress, tải file đúng tên/hash và cập nhật peers khi đăng xuất. Script có thể chạy lại bằng scripts/smoke_web_ui.py; ảnh minh họa lưu tại docs/images/web-ui-smoke.png.

Chạy từ root: cài requirements, chạy security.py một lần, chạy server.py; terminal khác chạy btl_btlmmt/web_ui/server_web.py --port 5000. Mở hai tab localhost để demo. Lệnh đầy đủ và kết quả kiểm thử ở docs/TESTING.md; đặc tả ở docs/PROTOCOL.md.

Script benchmark_clients.py tách server thành tiến trình riêng và chạy 20/40 client truyền file 1 MiB đồng thời theo vòng, kiểm tra hash và broadcast. Kết quả CPU/RAM, độ trễ và số luồng lưu ở docs/performance; phân tích ở docs/PERFORMANCE.md.

Giới hạn: kiểm thử trên Windows localhost, chưa benchmark WAN/LAN nhiều máy; CUBIC-style chưa có ECN/HyStart++/fast recovery; chưa resume file hoặc tài khoản mật khẩu. Không suy diễn điểm chấm từ số test. Báo cáo không khẳng định E2E hay full RFC compliance. Không đưa chứng thư riêng, downloads hoặc file tạm vào ZIP.